

1. Resolve Merge Conflicts

- a. Create a merge conflict intentionally (two users editing the same line).
- b. Resolve the conflict and push the changes.

Objective : Create a merge conflict intentionally by making two different changes to the same line from two branches, resolve the conflict, and push the final changes to GitHub.

Step 1: Create a File on Main Branch

```
git checkout main
```

```
vi conflict.txt
```

Add:

Hello World

Save and exit:

Esc

:wq

Enter

Commit and push:

```
git add conflict.txt
```

```
git commit -m "Added conflict file"
```

```
git push origin main
```

Step 2: Create a Feature Branch

```
git checkout -b feature
```

Edit the same line:

```
vi conflict.txt
```

Change it to:

Hello from User 1

Save and commit:

```
git add conflict.txt
```

```
git commit -m "Updated by User 1"
```

Step 3: Make Another Change on Main

Switch back to main:

```
git checkout main
```

Edit the same line:

```
vi conflict.txt
```

Change it to:

Hello from User 2

Save and commit:

```
git add conflict.txt
```

```
git commit -m "Updated by User 2"
```

Step 4: Create the Merge Conflict

Merge the feature branch:

```
git merge feature
```

Git will show a conflict because both branches changed the same line.

Check the conflict:

```
git status
```

```
cat conflict.txt
```

The file will look similar to:

```
<<<<<< HEAD
```

```
Hello from User 2
```

```
=====
```

```
Hello from User 1
```

```
>>>>>> feature
```

Step 5: Resolve the Conflict**Open the file:**

```
vi conflict.txt
```

Remove the conflict markers and keep the required final content:

```
Hello from User 1 and User 2
```

Save and exit:

```
Esc
```

```
:wq
```

```
Enter
```

Step 6: Complete the Merge**Tell Git that the conflict has been resolved:**

```
git add conflict.txt
```

```
git commit -m "Resolved merge conflict"
```

Step 7: Push the Changes to GitHub

```
git push origin main
```

Result

The merge conflict was successfully:

Created by changing the same line in two branches.

Detected by Git during the merge.

Resolved manually by editing the conflicted file.

Committed after resolving the conflict.

Pushed successfully to GitHub.

```

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ vi conflict.sh

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git add conflict.sh
warning: in the working copy of 'conflict.sh', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git commit -m "Added the conflict.sh file to the main"
[main ac7af22] Added the conflict.sh file to the main
1 file changed, 1 insertion(+)
create mode 100644 conflict.sh

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 294 bytes | 98.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/chvkarthik-87/devops-git-assignment.git
14ed980..ac7af22 main -> main

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git checkout -b feature
Switched to a new branch 'feature'

chvka@karthik MINGW64 ~/devops-git-assignment (feature)
$ vi conflict.sh

chvka@karthik MINGW64 ~/devops-git-assignment (feature)
$ git add conflict.sh
warning: in the working copy of 'conflict.sh', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (feature)
$ git commit -m "updated by user 1"
[feature de426bd] updated by user 1
1 file changed, 1 insertion(+), 1 deletion(-)

chvka@karthik MINGW64 ~/devops-git-assignment (feature)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

```

```

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ vi conflict.sh

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git add conflict.sh

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git commit -m "updated by user 2"
[main d119a5c] updated by user 2
1 file changed, 1 insertion(+), 1 deletion(-)

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git merge feature
Auto-merging conflict.sh
CONFLICT (content): Merge conflict in conflict.sh
Automatic merge failed; fix conflicts and then commit the result.

chvka@karthik MINGW64 ~/devops-git-assignment (main|MERGING)
$ cat conflict.sh
<<<<<< HEAD
Hello from user 2
=====
Hello from user 1
>>>>>> feature

chvka@karthik MINGW64 ~/devops-git-assignment (main|MERGING)
$ vi conflict.sh

chvka@karthik MINGW64 ~/devops-git-assignment (main|MERGING)
$ git add conflict.sh

chvka@karthik MINGW64 ~/devops-git-assignment (main|MERGING)
$ git commit -m "resolve merge conflict"
[main 931eb8c] resolve merge conflict

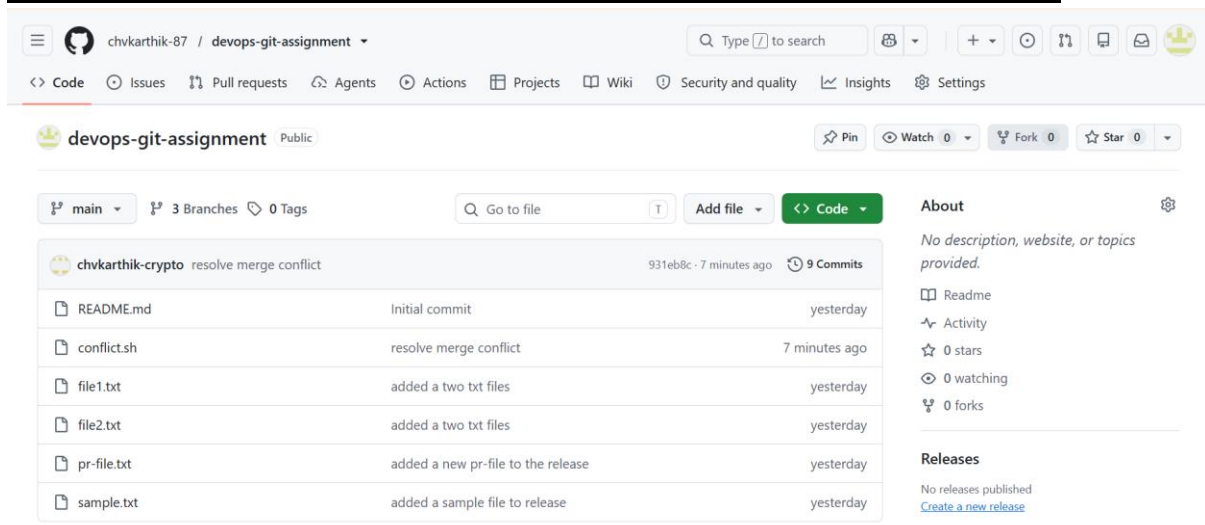
chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git add .

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 726 bytes | 363.00 KiB/s, done.
Total 9 (delta 4), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (4/4), completed with 1 local object.
To https://github.com/chvkarthik-87/devops-git-assignment.git

```

```
chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ cat conflict.sh

Hello from user 1 user 2
```



2. Recover Deleted Branch

a. Delete a local branch and then recover it using the reflog.

Objective :Delete a local branch accidentally and recover it using git reflog.

Step 1: Create a New Branch

```
git checkout -b recovery-test
```

Step 2: Create a File

```
vi recovery.txt
```

Add:

This branch needs to be recovered.

Save and exit:

```
Esc
```

```
:wq
```

```
Enter
```

Step 3: Commit the Changes

```
git add recovery.txt
```

```
git commit -m "Added recovery file"
```

Step 4: Go Back to Main

```
git checkout main
```

Step 5: Delete the Local Branch

```
git branch -D recovery-test
```

Check:

```
git branch
```

The recovery-test branch is now deleted.

Step 6: Find the Deleted Branch Using Reflog

git reflog

Find the commit that says:

commit: Added recovery file

Copy its commit ID.

For example:

abc1234

Step 7: Recover the Branch

git checkout -b recovery-test abc1234

Check:

git branch

You should see:

* recovery-test

main

Step 8: Verify the Recovered File

cat recovery.txt

Output:

This branch needs to be recovered.

Result :The deleted local branch was successfully recovered using git reflog.

```
chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git checkout -b recovery-test
Switched to a new branch 'recovery-test'

chvka@karthik MINGW64 ~/devops-git-assignment (recovery-test)
$ vi recovery.txt

chvka@karthik MINGW64 ~/devops-git-assignment (recovery-test)
$ git add recovery.txt
warning: in the working copy of 'recovery.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (recovery-test)
$ git commit -m "added recovery file"
[recovery-test 022ad7d] added recovery file
1 file changed, 1 insertion(+)
create mode 100644 recovery.txt

chvka@karthik MINGW64 ~/devops-git-assignment (recovery-test)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git branch -D recovery-test
Deleted branch recovery-test (was 022ad7d).

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git branch
  feature
* main

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git reflog
931eb8c (HEAD -> main, origin/main, origin/HEAD) HEAD@{0}: checkout: moving from recovery-test to main
022ad7d HEAD@{1}: commit: added recovery file
931eb8c (HEAD -> main, origin/main, origin/HEAD) HEAD@{2}: checkout: moving from main to recovery-test
931eb8c (HEAD -> main, origin/main, origin/HEAD) HEAD@{3}: commit (merge): resolve merge conflict
d119a5c HEAD@{4}: commit: updated by user 2
ac7af22 HEAD@{5}: checkout: moving from feature to main
de426bd (feature) HEAD@{6}: commit: updated by user 1
ac7af22 HEAD@{7}: checkout: moving from main to feature
ac7af22 HEAD@{8}: commit: Added the conflict.sh file to the main
14ed980 HEAD@{9}: clone: from https://github.com/chvkarthik-87/devops-git-assignment.git

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git checkout -b recovery-test 022ad7d
Switched to a new branch 'recovery-test'
```

```
chvka@karthik MINGW64 ~/devops-git-assignment (recovery-test)
$ git branch
  feature
  main
* recovery-test

chvka@karthik MINGW64 ~/devops-git-assignment (recovery-test)
$ cat recovery.txt
This branch needs to be recovered
```

3. Undo Wrong Push

- a. Push a wrong commit to GitHub, then undo it without losing history.

Objective : Push a wrong commit to GitHub and then undo it without deleting the commit from history.

We use git revert for this.

Step 1: Go to Main Branch

```
git checkout main
```

Check:

```
git status
```

Step 2: Create a Wrong Change

```
vi wrong.txt
```

Add:

This is a wrong change.

Save and exit:

Esc

:wq

Enter

Step 3: Commit the Wrong Change

```
git add wrong.txt
```

```
git commit -m "Added wrong change"
```

Step 4: Push the Wrong Commit to GitHub

```
git push origin main
```

Now the wrong commit is on GitHub.

Step 5: Undo the Wrong Commit

Use:

```
git revert HEAD
```

Git will open vi for the revert commit message.

Just save and exit:

Esc

:wq

Enter

Git creates a new commit that reverses the wrong change.

Step 6: Push the Revert Commit

```
git push origin main
```

Step 7: Check History

```
git log --oneline
```

You will see something similar to:

abc4567 Revert "Added wrong change"

abc1234 Added wrong change

The wrong commit is still in history, but its changes have been undone.

```
chvka@karthik MINGW64 ~/devops-git-assignment (recovery-test)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ vi wrong.txt

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git add wrong.txt
warning: in the working copy of 'wrong.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git commit -m "Added the wrong change to main"
[main 404b856] Added the wrong change to main
 1 file changed, 1 insertion(+)
 create mode 100644 wrong.txt

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 299 bytes | 149.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/chvkarthik-87/devops-git-assignment.git
 931eb8c..404b856  main -> main

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git revert HEAD
[main 6c2cf52] Revert "Added the wrong change to main"
 1 file changed, 1 deletion(-)
 delete mode 100644 wrong.txt

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 259 bytes | 259.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/chvkarthik-87/devops-git-assignment.git
 404b856..6c2cf52  main -> main

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git log --oneline
6c2cf52 (HEAD -> main, origin/main, origin/HEAD) Revert "Added the wrong change to main"
404b856 Added the wrong change to main
931eb8c resolve merge conflict
d119a5c updated by user 2
de426bd (feature) updated by user 1
ac7af22 Added the conflict.sh file to the main
14ed980 Merge pull request #1 from chvkarthik-87/release
fac54f2 added a new pr-file to the release
3a50b4f added a sample file to release
45192c6 (origin/github-branch) added a two txt files
2b74832 Initial commit
```

4. Amend a Commit

a. Make a commit, then add a missing file to it using `git commit --amend`.

Purpose : `git commit --amend` is used to modify the latest commit.

It is useful when we forgot to add a file or want to change the latest commit message.

Steps

First, create a file:

```
echo "Hello" > file1.txt
```

Add and commit it:

```
git add file1.txt
```

```
git commit -m "Added file1"
```

Now suppose we forgot another file:

```
echo "New file" > file2.txt
```

Add the missing file:

```
git add file2.txt
```

Add it to the previous commit:

```
git commit --amend --no-edit
```

Check the commit:

```
git log --oneline
```

Result : The new file is added to the previous commit instead of creating a new commit.

```
chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ vi file1.txt

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git add file1.txt
warning: in the working copy of 'file1.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git commit -m "added the file1.txt"
[main a2bdfa9] added the file1.txt
 2 files changed, 2 insertions(+)

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ vi file2.txt

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git add .
warning: in the working copy of 'file2.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git commit --amend --no-edit
[main 904a99a] added the file1.txt
 Date: Sat Aug 22 18:14:16 2026 +0530
 3 files changed, 3 insertions(+)

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git log --online
fatal: unrecognized argument: --online

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git log --oneline
904a99a (HEAD -> main) added the file1.txt
6c2cf52 (origin/main, origin/HEAD) Revert "Added the wrong change to main"
404b856 Added the wrong change to main
931eb8c resolve merge conflict
d119a5c updated by user 2
de426bd (feature) updated by user 1
ac7af22 Added the conflict.sh file to the main
14ed980 Merge pull request #1 from chvkarthik-87/release
fac54f2 added a new pr-file to the release
3a50b4f added a sample file to release
45192c6 (origin/github-branch) added a two txt files
2b74832 Initial commit
```

5. Cherry-pick a Commit

a. Take a specific commit from one branch and apply it to another branch.

Purpose :git cherry-pick is used to take one specific commit from one branch and apply it to another branch.

Steps

Create a new branch:

```
git checkout -b feature
```

Create a file:

```
echo "Feature work" > feature.txt
```

Commit the file:

```
git add feature.txt
```

```
git commit -m "Added feature"
```

Find the commit ID:

```
git log --oneline
```

Example:

```
a1b2c3d Added feature
```

Copy the commit ID.

Go to the main branch:

```
git checkout main
```

Apply the commit:

```
git cherry-pick a1b2c3d
```

Check the log:

```
git log --oneline
```

Result :The selected commit from the feature branch is applied to the main branch.

```

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git checkout feature
Switched to branch 'feature'

chvka@karthik MINGW64 ~/devops-git-assignment (feature)
$ ls
README.md  conflict.sh  file1.txt  file2.txt  pr-file.txt  sample.txt

chvka@karthik MINGW64 ~/devops-git-assignment (feature)
$ echo "feature work" > feature.txt

chvka@karthik MINGW64 ~/devops-git-assignment (feature)
$ git add feature.txt
warning: in the working copy of 'feature.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (feature)
$ git commit -m "added new file"
[feature 672511f] added new file
1 file changed, 1 insertion(+)
create mode 100644 feature.txt

chvka@karthik MINGW64 ~/devops-git-assignment (feature)
$ git log --oneline
672511f (HEAD -> feature) added new file
de426bd updated by user 1
ac7af22 Added the conflict.sh file to the main
14ed980 Merge pull request #1 from chvkarthik-87/release
fac54f2 added a new pr-file to the release
3a50b4f added a sample file to release
45192c6 (origin/github-branch) added a two txt files
2b74832 Initial commit

```

```

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git cherry-pick 672511f
[main 4066d29] added new file
Date: Sat Aug 22 18:23:10 2026 +0530
1 file changed, 1 insertion(+)
create mode 100644 feature.txt

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git log --oneline
4066d29 (HEAD -> main) added new file
904a99a added the file1.txt
6c2cf52 (origin/main, origin/HEAD) Revert "Added the wrong change to main"
404b856 Added the wrong change to main
931eb8c resolve merge conflict
d119a5c updated by user 2
de426bd updated by user 1
ac7af22 Added the conflict.sh file to the main
14ed980 Merge pull request #1 from chvkarthik-87/release
fac54f2 added a new pr-file to the release
3a50b4f added a sample file to release
45192c6 (origin/github-branch) added a two txt files
2b74832 Initial commit

```

6. Interactive Rebase

a. Reorder and squash multiple commits into a single clean commit.

Purpose :Interactive rebase is used to clean and organize multiple commits.

We can:

Reorder commits

Squash commits

Change commit messages

Steps

Create three commits:

```
echo "Line 1" > file.txt
```

```
git add file.txt
```

```
git commit -m "Commit 1"
```

```
echo "Line 2" >> file.txt
```

```
git add file.txt
```

```
git commit -m "Commit 2"
```

```
echo "Line 3" >> file.txt
```

```
git add file.txt
```

```
git commit -m "Commit 3"
```

Check the commits:

```
git log --oneline -3
```

Start interactive rebase:

```
git rebase -i HEAD~3
```

Git will show:

```
pick Commit1
```

```
pick Commit2
```

```
pick Commit3
```

To combine the commits, change it to:

```
pick Commit1
```

```
squash Commit2
```

```
squash Commit3
```

Save and close the editor.

Give the final commit message:

```
Combined commits
```

Check the log:

```
git log --oneline
```

Result :Three commits are combined into one clean commit.

```

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ echo "line 1" > file.txt

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git add file.txt
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git commit -m "commit 1"
[main 5bae078] commit 1
1 file changed, 1 insertion(+), 1 deletion(-)

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ echo "line 2" >> file.txt
> ^C

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ echo "line 2" >> file.txt

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git add file.txt
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git commit -m "commit 2"
[main 2642aec] commit 2
1 file changed, 1 insertion(+)

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ echo "line3" >> file.txt

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git add file.txt
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git commit -m "commit 3"
[main 6f8fb9a] commit 3
1 file changed, 1 insertion(+)

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git log --oneline -3
6f8fb9a (HEAD -> main) commit 3
2642aec commit 2
5bae078 commit 1

```

```

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git rebase -i HEAD~3
[detached HEAD 942dc84] commit 1
Date: Sat Aug 22 18:57:59 2026 +0530
1 file changed, 3 insertions(+), 1 deletion(-)
Successfully rebased and updated refs/heads/main.

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git log --oneline
942dc84 (HEAD -> main) commit 1
9602a2a commit 1
4066d29 added new file
904a99a added the file1.txt
6c2cf52 (origin/main, origin/HEAD) Revert "Added the wrong change to main"
404b856 Added the wrong change to main
931eb8c resolve merge conflict
d119a5c updated by user 2
de426bd updated by user 1
ac7af22 Added the conflict.sh file to the main
14ed980 Merge pull request #1 from chvkarthik-87/release
fac54f2 added a new pr-file to the release
3a50b4f added a sample file to release
45192c6 (origin/github-branch) added a two txt files
2b74832 Initial commit

```

```

pick 5bae078 commit 1
pick 2642aec commit 2
pick 6f8fb9a commit 3

# Rebase 9602a2a..6f8fb9a onto 9602a2a (3 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .       create a merge commit using the original merge commit's
# .       message (or the oneline, if no original merge commit was
# .       specified); use -c <commit> to reword the commit message
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
~
~

```

7. Tagging & Release

a. Create a version tag (v1.0), push it to GitHub, then delete and restore it.

Purpose :A Git tag is used to mark a specific version of the project.

For example:

v1.0

v2.0

v3.0

Tags are commonly used for software releases.

Step 1: Create a Tag

First make a commit:

git add .

git commit -m "Release version 1.0"

Create the tag:

git tag v1.0

Check the tag:

git tag

Output:

v1.0

Step 2: Push Tag to GitHub

git push origin v1.0

Now the v1.0 tag is available on GitHub.

Step 3: Delete the Local Tag

```
git tag -d v1.0
```

Check:

```
git tag
```

The tag is removed from the local repository.

Step 4: Delete the Tag from GitHub

```
git push origin --delete v1.0
```

Now the tag is also removed from GitHub.

Step 5: Restore the Tag

Create the tag again:

```
git tag v1.0
```

Push it to GitHub:

```
git push origin v1.0
```

Check:

```
git tag
```

Result :The v1.0 tag is created, pushed to GitHub, deleted, and restored successfully.

```

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git tag v1.0

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git tag
v1.0

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git push origin v1.0
Enumerating objects: 18, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 8 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (14/14), 1.07 KiB | 182.00 KiB/s, done.
Total 14 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/chvkarthik-87/devops-git-assignment.git
 * [new tag]          v1.0 -> v1.0

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git tag -d v1.0
Deleted tag 'v1.0' (was 942dc84)

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git tag

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git push origin --delete v1.0
To https://github.com/chvkarthik-87/devops-git-assignment.git
 - [deleted]          v1.0

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git tag v1.0

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git push origin v1.0
Enumerating objects: 18, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 8 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (14/14), 1.07 KiB | 273.00 KiB/s, done.
Total 14 (delta 3), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/chvkarthik-87/devops-git-assignment.git
 * [new tag]          v1.0 -> v1.0

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git tag
v1.0

```

8. Clone with Sparse Checkout

- a. Clone only a subdirectory of a repo using sparse checkout.

Purpose :Sparse checkout is used to clone a Git repository and check out only a specific subdirectory instead of the complete repository.

In this task, I used the "devops-git-assignment" repository and selected the "scripts" folder.

Step 1: Clone the Repository

First, clone the GitHub repository:

git clone <https://github.com/chvkarthik-87/devops-git-assignment.git>

Result :The repository was successfully cloned.

Cloning into 'devops-git-assignment'... Receiving objects: 100% Resolving deltas: 100% done.

Step 2: Go Inside the Repository

Move into the cloned repository:

```
cd devops-git-assignment
```

Check the current location:

```
pwd
```

Step 3: Check the Repository Contents

Run:

```
ls
```

Initially, the repository contained files such as:

```
README.md conflict.sh file1.txt file2.txt pr-file.txt sample.txt
```

There was no subdirectory available, so I created a "scripts" directory for this task.

Step 4: Create a Subdirectory

Create the "scripts" folder:

```
mkdir scripts
```

Create a file inside the folder:

```
echo "My script" > scripts/test.sh
```

Check the folder:

```
ls
```

Step 5: Add and Commit the Folder

Add the "scripts" folder:

```
git add scripts
```

Create a commit:

```
git commit -m "Added scripts folder"
```

Step 6: Push the Changes to GitHub

Push the commit to GitHub:

```
git push origin main
```

If the output says:

Everything up-to-date

it means there are no new changes to push.

Step 7: Enable Sparse Checkout

Enable sparse checkout using:

```
git sparse-checkout init --cone
```

The repository will show "SPARSE" in the Git Bash prompt.

Example:

```
(main | SPARSE)
```

This indicates that sparse checkout is enabled.

Step 8: Select Only the Required Folder

Select the "scripts" folder:

```
git sparse-checkout set scripts
```

Important

The correct command is:

```
git sparse-checkout
```

Do not type:

```
git spare-checkout
```

"spare-checkout" is incorrect.

Step 9: Verify the Sparse Checkout

Check which folder is selected:

```
git sparse-checkout list
```

Output:

```
scripts
```

This confirms that the "scripts" folder is selected.

Step 10: Check the Files

Run:

```
ls
```

The selected "scripts" directory should be available.

Check its contents:

```
ls scripts
```

Output:

```
test.sh
```

Result :The repository was successfully configured with sparse checkout, and only the required "scripts" subdirectory was checked out.

```

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git clone https://github.com/chvkarthik-87/devops-git-assignment.git
Cloning into 'devops-git-assignment'...
remote: Enumerating objects: 45, done.
remote: Counting objects: 100% (45/45), done.
remote: Compressing objects: 100% (21/21), done.
remote: Total 45 (delta 14), reused 37 (delta 10), pack-reused 0 (from 0)
Receiving objects: 100% (45/45), 5.04 KiB | 368.00 KiB/s, done.
Resolving deltas: 100% (14/14), done.

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ pwd
/c/Users/chvka/devops-git-assignment

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ ls
README.md  conflict.sh  devops-git-assignment/  feature.txt  file.txt  file1.txt  file2.txt  pr-file.txt  sample.txt

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ cd devops-git-assignment/

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main)
$ pwd
/c/Users/chvka/devops-git-assignment/devops-git-assignment

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main)
$ git sparse-checkout init --cone

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ ls
README.md  conflict.sh  file1.txt  file2.txt  pr-file.txt  sample.txt

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ mkdir scripts

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ echo "my scripts" > scripts/test.sh

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ git add scripts

Your branch is up to date with 'origin/main'.

You are in a sparse checkout with 100% of tracked files present.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    scripts/

nothing added to commit but untracked files present (use "git add" to track)

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ git push origin main
Everything up-to-date

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ git sparse-checkout set scripts
git: 'sparse-checkout' is not a git command. See 'git --help'.

The most similar command is
  sparse-checkout

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ git spares-checkout set scripts
git: 'spares-checkout' is not a git command. See 'git --help'.

The most similar command is
  sparse-checkout

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ git sparese-checkout set scripts
git: 'sparese-checkout' is not a git command. See 'git --help'.

The most similar command is
  sparse-checkout

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ git sparse-checkout set scripts

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ git sparse-checkout list
scripts

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ ls
README.md  conflict.sh  file1.txt  file2.txt  pr-file.txt  sample.txt  scripts/

```

9. Reset vs Revert Challenge

a. Demonstrate the difference between `git reset --hard` and `git revert` in a repo.

Purpose : `git reset` and `git revert` are both used to undo changes in Git.

The main difference is:

Reset changes the existing Git history.

Revert creates a new commit that reverses the changes of an existing commit.

Part A: Git Reset

Step 1: Create a Test Branch

To avoid changing the main branch, create a separate branch:

```
git checkout -b reset-revert-test
```

Step 2: Create a Test File

```
echo "Reset test" > reset.txt
```

Step 3: Add the File

```
git add reset.txt
```

Step 4: Commit the File

```
git commit -m "added reset test"
```

The commit created was:

```
13d0372 added reset test
```

Step 5: Check the Commit History

```
git log --oneline -3
```

Output:

```
13d0372 added reset test
```

```
0884c08 added the new scripts
```

```
6c2cf52 Revert "Added the wrong change to main"
```

Step 6: Reset the Commit

We used the previous commit:

```
git reset --hard 0884c08
```

Output:

```
HEAD is now at 0884c08 added the new scripts
```

Step 7: Verify the Reset

```
git log --oneline -3
```

The 13d0372 added reset test commit was no longer present in the current branch history.

Result of Reset

The reset moved the branch back to commit 0884c08.

Before Reset:

```
C1 ---- C2 ---- C3
```

```
↑
```

```
reset test
```

After Reset:

C1 ---- C2

Therefore, Git Reset changes the existing branch history.

Part B: Git Revert

Step 1: Create a Test File

```
echo "Revert test" > revert.txt
```

Step 2: Add the File

```
git add revert.txt
```

Step 3: Commit the File

```
git commit -m "added revert test"
```

The commit created was:

9fa9af6 added revert test

Step 4: Check the Commit History

```
git log --oneline -3
```

Step 5: Revert the Commit

```
git revert 9fa9af6
```

Git opened the commit message editor and showed:

Revert "added revert test"

This reverts commit 9fa9af6.

After saving and closing the editor, Git created a new revert commit.

Step 6: Check the History

```
git log --oneline -4
```

Output:

531585e Revert "added revert test"

9fa9af6 added revert test

0884c08 added the new scripts

6c2cf52 Revert "Added the wrong change to main"

This proves that the original commit 9fa9af6 is still present in the history, and Git created a new commit 531585e to undo its changes.

Result of Revert

Before Revert:

C1 ---- C2

↑

original commit

After Revert:

C1 ---- C2 ---- C3

↑ ↑

original revert

Therefore, Git Revert does not remove the original commit. It creates a new commit that reverses the changes.

```

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

You are in a sparse checkout with 100% of tracked files present.

nothing to commit, working tree clean

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ git log --oneline
0884c08 (HEAD -> main) added the new scripts
6c2cf52 (origin/main, origin/HEAD) Revert "Added the wrong change to main"
404b856 Added the wrong change to main
931eb8c resolve merge conflict
d119a5c updated by user 2
de426bd updated by user 1
ac7af22 Added the conflict.sh file to the main
14ed980 Merge pull request #1 from chvkarthik-87/release
fac54f2 added a new pr-file to the release
3a50b4f added a sample file to release
45192c6 (origin/github-branch) added a two txt files
2b74832 Initial commit

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (main|SPARSE)
$ git checkout -b reset-revert-test
Switched to a new branch 'reset-revert-test'

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git branch
  main
* reset-revert-test

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ echo "reset test" > reset.txt

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git add reset.txt
warning: in the working copy of 'reset.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git commit -m "added reset test"
[reset-revert-test 13d0372] added reset test
 1 file changed, 1 insertion(+)
 create mode 100644 reset.txt

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git log --oneline -3
13d0372 (HEAD -> reset-revert-test) added reset test
0884c08 (main) added the new scripts
6c2cf52 (origin/main, origin/HEAD) Revert "Added the wrong change to main"

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git reset --hard 0884c08
HEAD is now at 0884c08 added the new scripts

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git log --oneline -3
0884c08 (HEAD -> reset-revert-test, main) added the new scripts
6c2cf52 (origin/main, origin/HEAD) Revert "Added the wrong change to main"
404b856 Added the wrong change to main

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git reset --hard 0884c08
HEAD is now at 0884c08 added the new scripts

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ echo "revert test" > revert.txtz

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git commit -m "added revert test"
On branch reset-revert-test
You are in a sparse checkout with 100% of tracked files present.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  revert.txtz

nothing added to commit but untracked files present (use "git add" to track)

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git add revert.txtz
warning: in the working copy of 'revert.txtz', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git commit -m "added revert test"
[reset-revert-test 9fa9af6] added revert test
 1 file changed, 1 insertion(+)
 create mode 100644 revert.txtz

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git log --oneline -3
9fa9af6 (HEAD -> reset-revert-test) added revert test
0884c08 (main) added the new scripts

```

```

0884c08 (main) added the new scripts
6c2cf52 (origin/main, origin/HEAD) Revert "Added the wrong change to main"

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git revert 9fa9af6
[reset-revert-test 531585e] Revert "added revert test"
1 file changed, 1 deletion(-)
delete mode 100644 revert.txtz

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git log --oneline -4
531585e (HEAD -> reset-revert-test) Revert "added revert test"
9fa9af6 added revert test
0884c08 (main) added the new scripts
6c2cf52 (origin/main, origin/HEAD) Revert "Added the wrong change to main"

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ ls
README.md  conflict.sh  file1.txt  file2.txt  pr-file.txt  sample.txt  scripts/

```

10. Detached HEAD Challenge

- Checkout a specific commit (detached HEAD state) and create a new branch from it.

Goal: Checkout a specific commit, enter Detached HEAD state, and then create a new branch from that commit.

We can use your existing commit 0884c08.

Step 1: Check your current branch

Run:

```
git branch
```

You are currently on:

```
reset-revert-test
```

Step 2: Check your commit history

```
git log --oneline
```

You already have:

```
0884c08 added the new scripts
```

We will use 0884c08.

Step 3: Checkout the specific commit

Run:

```
git checkout 0884c08
```

You should see a message similar to:

You are in 'detached HEAD' state

Step 4: Verify Detached HEAD

Run:

```
git status
```

You should see:

```
HEAD detached at 0884c08
```

This means HEAD is pointing directly to a commit, not to a branch.

Step 5: Create a new branch

Now create a branch from this commit:

```
git checkout -b detached-feature
```

You should see:

Switched to a new branch 'detached-feature'

Step 6: Verify the branch

Run:

```
git branch
```

You should see something like:

```
* detached-feature
```

```
main
```

```
reset-revert-test
```

The * shows your current branch.

Step 7: Verify the commit

Run:

```
git log --oneline -3
```

You should see:

0884c08 added the new scripts

Simple Explanation

Commit History

C1 ---- C2 ---- C3

↑

0884c08

|

Detached HEAD

|

↓

detached-feature

Result :The specific commit 0884c08 was checked out in Detached HEAD state, and a new branch named detached-feature was successfully created from that commit.

```

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git branch
  main
* reset-revert-test

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git log --oneline
531585e (HEAD -> reset-revert-test) Revert "added revert test"
9fa9af6 added revert test
0884c08 (main) added the new scripts
6c2cf52 (origin/main, origin/HEAD) Revert "Added the wrong change to main"
404b856 Added the wrong change to main
931eb8c resolve merge conflict
d119a5c updated by user 2
de426bd updated by user 1
ac7af22 Added the conflict.sh file to the main
14ed980 Merge pull request #1 from chvkarthik-87/release
fac54f2 added a new pr-file to the release
3a50b4f added a sample file to release
45192c6 (origin/github-branch) added a two txt files
2b74832 Initial commit

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (reset-revert-test|SPARSE)
$ git checkout 0884c08
Note: switching to '0884c08'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

    git switch -c <new-branch-name>

Or undo this operation with:

    git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at 0884c08 added the new scripts

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment ((0884c08...)|SPARSE)
$ git status
HEAD detached at 0884c08
You are in a sparse checkout with 100% of tracked files present.
You are in a sparse checkout with 100% of tracked files present.

nothing to commit, working tree clean

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment ((0884c08...)|SPARSE)
$ git checkout -b detached-feature
Switched to a new branch 'detached-feature'

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ git branch
* detached-feature
  main
  reset-revert-test

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ git log --oneline -3
0884c08 (HEAD -> detached-feature, main) added the new scripts
6c2cf52 (origin/main, origin/HEAD) Revert "Added the wrong change to main"
404b856 Added the wrong change to main

```

11. Git Hooks Challenge

- Configure a pre-commit hook to reject commits without a message format (e.g., must start with JIRA-XXX).

Purpose : A Git Hook is used to automatically perform an action before or after a Git command.

In this task, we created a commit-msg hook that accepts only commit messages starting with IRA-XXX.

Example:

☒ Valid:

IRA-101 Added new file

☐ Invalid:

Added new file

Step 1: Go to Git Hooks folder

cd .git/hooks

Step 2: Create the hook

```
touch commit-msg
```

Step 3: Open the hook

```
nano commit-msg
```

Add:

```
#!/bin/bash
```

```
commit_msg=$(cat "$1")
```

```
if [[ ! "$commit_msg" =~ ^IRA-[0-9]+ ]]; then  
    echo "ERROR: Commit message must start with IRA-XXX"  
    exit 1  
fi
```

Save and exit:

Ctrl + O → Enter → Ctrl + X

Step 4: Give permission

```
chmod 755 commit-msg
```

Step 5: Go back to repository

```
cd ../..
```

Step 6: Create a test file

```
echo "hook test" > hook-tst.txt
```

Step 7: Add the file

```
git add hook-tst.txt
```

Step 8: Test with an invalid message

```
git commit -m "added hook test"
```

Output:

ERROR: Commit message must start with IRA-XXX

The commit is rejected. ✖

Step 9: Test with a valid message

```
git commit -m "IRA-101 Added hook test"
```

The commit is successful. ☑

Step 10: Verify

```
git log --oneline -2
```

Example:

```
ef28fd8 IRA-101 Added hook test
```

```
0884c08 added the new scripts
```

Result

The commit-msg hook was successfully created.

Wrong format → Rejected ❌

IRA-XXX format → Accepted ✅

```
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ git status
On branch detached-feature
You are in a sparse checkout with 100% of tracked files present.

nothing to commit, working tree clean

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ cd .git/hooks
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment/.git/hooks (GIT_DIR!|SPARSE)
$ ls
applypatch-msg.sample*  fsmonitor-watchman.sample*  pre-applypatch.sample*  pre-merge-commit.sample*  pre-rebase.sample*  prepare-commit-msg.sample*  update.sample*
commit-msg.sample*      post-update.sample*          pre-commit.sample*      pre-push.sample*          pre-receive.sample*  push-to-checkout.sample*
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment/.git/hooks (GIT_DIR!|SPARSE)
$ touch commit-msg
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment/.git/hooks (GIT_DIR!|SPARSE)
$ ls
applypatch-msg.sample*  fsmonitor-watchman.sample*  pre-commit.sample*  pre-merge-commit.sample*  pre-rebase.sample*  push-to-checkout.sample*
commit-msg              post-update.sample*         pre-push.sample*    pre-receive.sample*       update.sample*
commit-msg.sample*      pre-applypatch.sample*      pre-push.sample*    prepare-commit-msg.sample*
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment/.git/hooks (GIT_DIR!|SPARSE)
$ nano commit-msg
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment/.git/hooks (GIT_DIR!|SPARSE)
$ chmod 755 commit-msg
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment/.git/hooks (GIT_DIR!|SPARSE)
$ cd ../../
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ echo "hook test" > hook-tst.txt
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ git add hook-tst.txt
warning: in the working copy of 'hook-tst.txt', LF will be replaced by CRLF the next time Git touches it
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ git commit -m "added hook test"
ERROR: Commit message must start with IRA-XXX
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ git commit -m "IRA-101 Added hook test"
[detached-feature ef28fd8] IRA-101 Added hook test
1 file changed, 1 insertion(+)
create mode 100644 hook-tst.txt
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ git log --oneline -2
ef28fd8 (HEAD -> detached-feature) IRA-101 Added hook test
0884c08 (main) added the new scripts
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ git commit -m "added hook test"
On branch detached-feature
You are in a sparse checkout with 100% of tracked files present.

nothing to commit, working tree clean

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ echo "hook test" > hook-tst.txt
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ git add hook-tst.txt
warning: in the working copy of 'hook-tst.txt', LF will be replaced by CRLF the next time Git touches it
chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ git commit -m "added hook test"
On branch detached-feature
You are in a sparse checkout with 100% of tracked files present.

nothing to commit, working tree clean

chvka@karthik MINGW64 ~/devops-git-assignment/devops-git-assignment (detached-feature|SPARSE)
$ ls -l hook-tst.txt
-rw-r--r-- 1 chvka 197609 10 Aug 22 20:26 hook-tst.txt
```

12. Squash Merge vs Rebase Merge

a. Show the difference between squash merge and rebase merge with evidence.

Objective : To demonstrate the difference between Squash Merge and Rebase Merge.

Squash Merge: Combines multiple commits into one commit.

Rebase Merge: Keeps commits separate and creates a clean, linear history.

Part 1: Squash Merge

Step 1: Create a feature branch

git checkout -b squash-feature

Step 2: Create the first commit

echo "Change 1" > squash.txt

```
git add squash.txt
```

```
git commit -m "Change 1"
```

Step 3: Create the second commit

```
echo "Change 2" >> squash.txt
```

```
git add squash.txt
```

```
git commit -m "Change 2"
```

Step 4: Create the third commit

```
echo "Change 3" >> squash.txt
```

```
git add squash.txt
```

```
git commit -m "Change 3"
```

Step 5: Check the commits

```
git log --oneline
```

We can see three separate commits:

Change 3

Change 2

Change 1

Step 6: Switch to main

```
git checkout main
```

Step 7: Perform Squash Merge

```
git merge --squash squash-feature
```

Step 8: Create one commit

```
git commit -m "Squashed squash-feature"
```

Result

The three commits are combined into one commit.

Change 1

Change 2 → One Commit

Change 3

Part 2: Rebase Merge

Step 1: Create a new branch

```
git checkout -b rebase-feature
```

Step 2: Create the first commit

```
echo "Rebase 1" > rebase.txt
```

```
git add rebase.txt
```

```
git commit -m "Rebase change 1"
```

Step 3: Create the second commit

```
echo "Rebase 2" >> rebase.txt
```

```
git add rebase.txt
```

```
git commit -m "Rebase change 2"
```

Step 4: Rebase with main

```
git rebase main
```

Output:

Current branch rebase-feature is up to date.

Step 5: Check the history

```
git log --oneline --graph --all
```

Example:

```
24417a6 Rebase change 2
```

```
d816d25 Rebase change 1
```

The commits remain separate.

```

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ echo checkout -b squash-features
checkout -b squash-features

chvka@karthik MINGW64 ~/devops-git-assignment (main)
$ git checkout -b squash-features
Switched to a new branch 'squash-features'

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ echo "change 1" > squash.txt

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ git add squash.txt
warning: in the working copy of 'squash.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ git commit -m "change 1"
[squash-features ff53689] change 1
1 file changed, 1 insertion(+)
create mode 100644 squash.txt

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ echo "change 2" >> squash.txt

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ git add squash.txt
warning: in the working copy of 'squash.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ git commit -m "change 2"
[squash-features ac2c79e] change 2
1 file changed, 1 insertion(+)

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ echo "change 3" >> squash.txt

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ git add squash.txt
warning: in the working copy of 'squash.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ git commit -m "change 3"
[squash-features f741839] change 3
1 file changed, 1 insertion(+)

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ git log --oneline -4
ac2c79e change 2
ff53689 change 1
942dc84 (tag: v1.0, main) commit 1

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ git merge --squash squash-features
Already up to date. (nothing to squash)

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ git commit -m "squashed squash-features"
On branch squash-features
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    devops-git-assignment/

nothing added to commit but untracked files present (use "git add" to track)

chvka@karthik MINGW64 ~/devops-git-assignment (squash-features)
$ git checkout -b rebase-feature
Switched to a new branch 'rebase-feature'

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ echo "rebase 1" > squash.txt

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ ls
README.md  conflict.sh  devops-git-assignment/  feature.txt  file.txt  file1.txt  file2.txt  pr-file.txt  sample.txt  squash.txt

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ rm squash.txt

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ echo "rebase 1" > rebase.txt

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ git add rebase.txt
warning: in the working copy of 'rebase.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ git commit -m "rebase change 1"
[rebase-feature d816d25] rebase change 1
1 file changed, 1 insertion(+)
create mode 100644 rebase.txt

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ echo "rebase 2" >> rebase.txt

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)

```

```

$ echo "rebase 2" >> rebase.txt

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ git add rebase.txt
warning: in the working copy of 'rebase.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ git commit -m "rebase change 2"
[rebase-feature 24417a6] rebase change 2
1 file changed, 1 insertion(+)

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ git rebase main
Current branch rebase-feature is up to date.

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ git log --oneline --graph --all -8
* 24417a6 (HEAD -> rebase-feature) rebase change 2
* d816d25 rebase change 1
* f741839 (squash-features) change 3
* ac2c79e change 2
* ff53689 change 1
* 942dc84 (tag: v1.0, main) commit 1
* 9602a2a commit 1
* 4066d29 added new file

```

13. Fork & Pull Request Workflow

a. Fork a repo, make a change, and submit a pull request to the original repo.

Objective : To demonstrate the Fork and Pull Request workflow using GitHub.

Step 1: Open a Public Repository

Opened a public GitHub repository:

GitHub for Beginners

Repository owner:

nisalgunawardhana

Step 2: Fork the Repository

Clicked the Fork button and created a copy of the repository under my GitHub account.

Forked repository:

chvkarthik-87/Github-for-beginners

Step 3: Clone the Fork

Copied the HTTPS URL of my fork and cloned it using Git Bash:

```
git clone https://github.com/chvkarthik-87/Github-for-beginners.git
```

Moved into the repository:

```
cd Github-for-beginners
```

Step 4: Create a New Branch

Created a new branch:

```
git checkout -b feature-change
```

Step 5: Make a Change

Created a new test file:

```
echo "fork and pull request test" > pr-test.txt
```

Step 6: Add the File

```
git add pr-test.txt
```

Step 7: Commit the Change

```
git commit -m "added pr test"
```

Step 8: Push the Branch

```
git push origin feature-change
```

The feature-change branch was successfully pushed to my GitHub fork.

Step 9: Create Pull Request

Opened my fork on GitHub and clicked:

Compare & pull request

The Pull Request was created from:

chvkarthik-87:feature-change

to:

nisalgunawardhana:main

Pull Request title:

Added PR test

Description:

Added a test file to demonstrate the Fork and Pull Request workflow.

Result

The Fork & Pull Request workflow was successfully demonstrated.

The following steps were completed:

Forked a public repository

Cloned the fork

Created a new branch

Created and committed a file

Pushed the branch to GitHub

Created a Pull Request to the original repository

The Pull Request could not be merged because the original repository has branch protection enabled. However, the Pull Request itself was successfully created.

```

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ git clone https://github.com/chvkarthik-87/Github-for-beginners.git
Cloning into 'Github-for-beginners'...
remote: Enumerating objects: 241, done.
remote: Counting objects: 100% (91/91), done.
remote: Compressing objects: 100% (17/17), done.
remote: Total 241 (delta 78), reused 74 (delta 74), pack-reused 150 (from 1)
Receiving objects: 100% (241/241), 10.55 MiB | 17.88 MiB/s, done.
Resolving deltas: 100% (104/104), done.

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ cd Github-for-beginners/

chvka@karthik MINGW64 ~/devops-git-assignment/Github-for-beginners (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

chvka@karthik MINGW64 ~/devops-git-assignment/Github-for-beginners (main)
$ git checkout -b feature-change
Switched to a new branch 'feature-change'

chvka@karthik MINGW64 ~/devops-git-assignment/Github-for-beginners (feature-change)
$ echo "Fork and pull request test" > pr-test.txt

chvka@karthik MINGW64 ~/devops-git-assignment/Github-for-beginners (feature-change)
$ git add pr-test.txt
warning: in the working copy of 'pr-test.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment/Github-for-beginners (feature-change)
$ git commit -m "added pr test"
[feature-change 0a5caca] added pr test
1 file changed, 1 insertion(+)
create mode 100644 pr-test.txt

chvka@karthik MINGW64 ~/devops-git-assignment/Github-for-beginners (feature-change)
$ git push origin feature-change
remote: Permission to chvkarthik-87/Github-for-beginners.git denied to chvkarthik-crypto.
fatal: unable to access 'https://github.com/chvkarthik-87/Github-for-beginners.git/': The requested URL returned error: 403

chvka@karthik MINGW64 ~/devops-git-assignment/Github-for-beginners (feature-change)
$ git remote -v
origin https://github.com/chvkarthik-87/Github-for-beginners.git (fetch)
origin https://github.com/chvkarthik-87/Github-for-beginners.git (push)

```

```

chvka@karthik MINGW64 ~/devops-git-assignment/Github-for-beginners (feature-change)
$ git push origin feature-change
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 290 bytes | 290.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature-change' on GitHub by visiting:
remote:   https://github.com/chvkarthik-87/Github-for-beginners/pull/new/feature-change
remote:
To https://github.com/chvkarthik-87/Github-for-beginners.git
 * [new branch]      feature-change -> feature-change

```

The screenshot shows a web browser window with a GitHub pull request page. The browser's address bar shows the URL: `github.com/nisalgunawardhana/Github-for-beginners/pull/1059`. The page title is "added pr test #1059". Below the title, it says "chvkarthik-87 wants to merge 1 commit into nisalgunawardhana:main from chvkarthik-87:feature-change". There are tabs for "Conversation", "Commits", "Checks", and "Files changed". The "Conversation" tab is active, showing a comment from "chvkarthik-87" that says "Added a test file to demonstrate the Fork and Pull Request workflow". Below the comment, there is a commit titled "added pr test" by "0a5caca". At the bottom, a status bar indicates "Some checks haven't completed yet" with "1 in progress check". On the right side, there are sections for "Reviewers" (No reviews), "Assignees" (No one assigned), "Labels" (None yet), and "Projects" (None yet).

14. Recover Lost Commit

a. Commit something, reset hard, and then recover it using `git reflog`.

Purpose : `git reflog` is used to find previous positions of HEAD, including commits that are no longer visible in the normal Git history.

Step 1: Create a Recovery Branch

Created a new branch:

```
git checkout -b recover-test
```

Step 2: Create and Commit a File

Created a file:

```
echo "Lost commit test" > recover.txt
```

Added the file:

```
git add recover.txt
```

Committed the file:

```
git commit -m "added recover test"
```

The commit was created with the ID:

```
b5305af added recover test
```

Checked the commit history:

```
git log --oneline -3
```

Step 3: Remove the Commit Using Reset

Intentionally removed the latest commit:

```
git reset --hard HEAD~1
```

The added recover test commit was no longer visible in the normal Git log.

Step 4: Find the Lost Commit

Used `git reflog`:

```
git reflog
```

The reflog showed the previous commit:

```
b5305af HEAD@{1}: commit: added recover test
```

The commit ID `b5305af` was identified as the lost commit.

Step 5: Recover the Commit

Recovered the lost commit using:

```
git reset --hard b5305af
```

Step 6: Verify the Recovery

Checked the commit history:

```
git log --oneline -3
```

The recovered commit appeared again:

```
b5305af (HEAD -> recover-test) added recover test
```

Also checked the files:

```
ls
```

The recover.txt file was present again.

Result

The commit that was removed using `git reset --hard` was successfully recovered using `git reflog`.

```

chvka@karthik MINGW64 ~/devops-git-assignment (rebase-feature)
$ git checkout -b recover-test
Switched to a new branch 'recover-test'

chvka@karthik MINGW64 ~/devops-git-assignment (recover-test)
$ echo "lost commit test" > recover.txt

chvka@karthik MINGW64 ~/devops-git-assignment (recover-test)
$ git add recover.txt
warning: in the working copy of 'recover.txt', LF will be replaced by CRLF the next time Git touches it

chvka@karthik MINGW64 ~/devops-git-assignment (recover-test)
$ git commit -m "added recover test"
[recover-test b5305af] added recover test
1 file changed, 1 insertion(+)
create mode 100644 recover.txt

chvka@karthik MINGW64 ~/devops-git-assignment (recover-test)
$ git log --oneline -3
b5305af (HEAD -> recover-test) added recover test
24417a6 (rebase-feature) rebase change 2
d816d25 rebase change 1

chvka@karthik MINGW64 ~/devops-git-assignment (recover-test)
$ git reset --hard HEAD~1
HEAD is now at 24417a6 rebase change 2

chvka@karthik MINGW64 ~/devops-git-assignment (recover-test)
$ git log --oneline -3
24417a6 (HEAD -> recover-test, rebase-feature) rebase change 2
d816d25 rebase change 1
f741839 (squash-features) change 3

chvka@karthik MINGW64 ~/devops-git-assignment (recover-test)
$ git reflog
24417a6 (HEAD -> recover-test, rebase-feature) HEAD@{0}: reset: moving to HEAD~1
b5305af HEAD@{1}: commit: added recover test
24417a6 (HEAD -> recover-test, rebase-feature) HEAD@{2}: checkout: moving from rebase-feature to recover-test
24417a6 (HEAD -> recover-test, rebase-feature) HEAD@{3}: commit: rebase change 2
d816d25 HEAD@{4}: commit: rebase change 1
f741839 (squash-features) HEAD@{5}: checkout: moving from squash-features to rebase-feature
f741839 (squash-features) HEAD@{6}: commit: change 3
ac2c79e HEAD@{7}: commit: change 2
ff53689 HEAD@{8}: commit: change 1
942dc84 (tag: v1.0, main) HEAD@{9}: checkout: moving from main to squash-features
942dc84 (tag: v1.0, main) HEAD@{10}: rebase (finish): returning to refs/heads/main
942dc84 (tag: v1.0, main) HEAD@{11}: rebase (squash): commit 1

8b5967e HEAD@{12}: rebase (squash): # This is a combination of 2 commits.
5bae078 HEAD@{13}: rebase (start): checkout HEAD~3
6f8fb9a HEAD@{14}: commit: commit 3
2642aec HEAD@{15}: commit: commit 2
5bae078 HEAD@{16}: commit: commit 1
9602a2a HEAD@{17}: commit: commit 1
4066d29 HEAD@{18}: cherry-pick: added new file
904a99a HEAD@{19}: checkout: moving from feature to main
672511f (feature) HEAD@{20}: commit: added new file
de426bd HEAD@{21}: checkout: moving from main to feature
904a99a HEAD@{22}: commit (amend): added the file1.txt
a2bdfa9 HEAD@{23}: commit: added the file1.txt
6c2cf52 (origin/main, origin/HEAD) HEAD@{24}: revert: Revert "Added the wrong change to main"
404b856 HEAD@{25}: commit: Added the wrong change to main
931eb8c HEAD@{26}: checkout: moving from recovery-test to main
022ad7d (recovery-test) HEAD@{27}: checkout: moving from main to recovery-test
931eb8c HEAD@{28}: checkout: moving from recovery-test to main
022ad7d (recovery-test) HEAD@{29}: commit: added recovery file
931eb8c HEAD@{30}: checkout: moving from main to recovery-test
931eb8c HEAD@{31}: commit (merge): resolve merge conflict
d119a5c HEAD@{32}: commit: updated by user 2
ac7af22 HEAD@{33}: checkout: moving from feature to main
de426bd HEAD@{34}: commit: updated by user 1
ac7af22 HEAD@{35}: checkout: moving from main to feature
ac7af22 HEAD@{36}: commit: Added the conflict.sh file to the main
14ed980 HEAD@{37}: clone: from https://github.com/chvkarthik-87/devops-git-assignment.git

chvka@karthik MINGW64 ~/devops-git-assignment (recover-test)
$ git reset --hard b5305af
HEAD is now at b5305af added recover test

chvka@karthik MINGW64 ~/devops-git-assignment (recover-test)
$ git log --oneline -3
b5305af (HEAD -> recover-test) added recover test
24417a6 (rebase-feature) rebase change 2
d816d25 rebase change 1

chvka@karthik MINGW64 ~/devops-git-assignment (recover-test)
$ ls
Github-for-beginners/  conflict.sh  feature.txt  file1.txt  pr-file.txt  recover.txt  squash.txt
README.md              devops-git-assignment/  file.txt    file2.txt  rebase.txt  sample.txt

chvka@karthik MINGW64 ~/devops-git-assignment (recover-test)
$

```